Universidad Da Vinci de Guatemala

Facultad de ingeniería, industria y tecnología

Ingeniería en sistemas

Ing. Brandon Antony Chitay Coutiño

Evaluación parcial 1

Erick Fernando Ortiz Pérez

Estructura de datos

202505618

03/03/2026

Introducción

Siguiendo las métricas establecidas para el tamaño de los datos, utilizaremos 4 tipos de ordenamientos algorítmicos para medir su tiempo y con ayuda de la notación Big-O veremos su tipo de crecimiento, si es lineal, constante, cuadrática o exponencial.

Haremos la comparación de todos los resultados para poder demostrar que no todos los enfoques son igual de eficientes. Esta comparación es muy importante para saber porque en algunos casos la versión iterativa es más rápida que la recursiva.

Algoritmos de ordenamiento

Factorial:

Cuando tenemos una lista de números y queremos encontrar la factorial, este algoritmo toma el primer numero de la lista, calcula su factorial multiplicando desde 1 hasta ese número y guarda o muestra el resultado luego pasa al siguiente numero de la lista y repite el proceso hasta terminar con todos.

En el caso iterativo se utiliza un ciclo for que multiplica acumulativamente los valores.

En recursividad utilizamos un caso base (0! =1) y una llamada recursiva n1 = n* (n-1)!

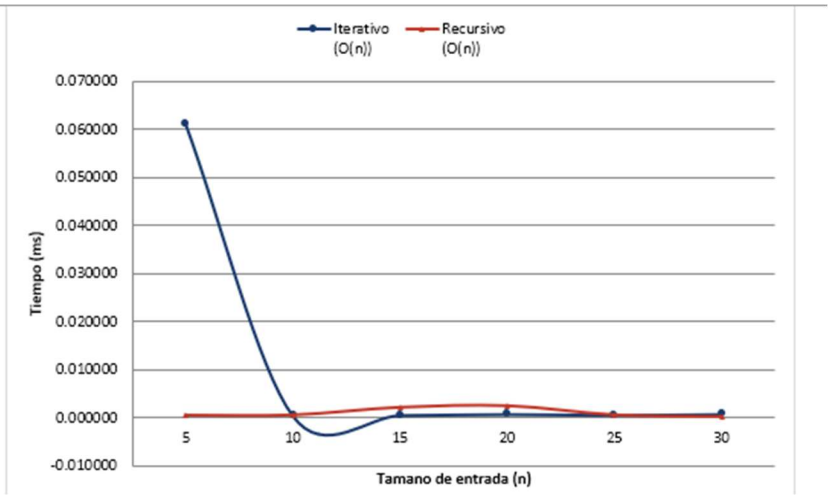
Su complejidad temporal es O(n)

La versión recursiva es mas elegante y refleja la definición matemática. Pero hay que tomar en cuenta que puede haber desbordamiento de pila si n es muy grande.

```java
package algorithms;

public class Factorial {

    // Versión iterativa O(n)
    public static long factorialIterativo(int n) {
        long resultado = 1;
        for (int i = 1; i <= n; i++) {
            resultado *= i;
        }
        return resultado;
    }

    // Versión recursiva O(n)
    public static long factorialRecursivo(int n) {
        if (n == 0)
            return 1;
        return n * factorialRecursivo(n - 1);
    }
}
```

n	Iterativo (O(n))	Recursivo (O(n))
5	0.061270	0.000518
10	0.000374	0.000562
15	0.000479	0.002166
20	0.000701	0.002488
25	0.000518	0.000578
30	0.000696	0.000234

Observacion: Ambas versiones son O(n): el tiempo crece linealmente con n.



Como podemos observar e n la tabla de algoritmo factorial, tiene un inicio más lento el iterativo comparado con el recursivo en este caso, por lo que será mucho mas conveniente utilizar la versión iterativa, aunque debemos tomar en cuenta que en las siguientes ejecuciones con valores mas grandes a 5 la diferencia no es mucha.

Fibonacci:

La serie Fibonacci comienza con 0 y 1, y cada número siguiente es la suma de los dos anteriores.

En el caso iterativo se utilizan variables acumuladoras como lo son a, b y c en un ciclo. Mientras que en el caso recursivo se utiliza fib(n)= fib(n-1) +fib(n-2), en este caso las complejidades cambian siendo iterativo O(n) y recursivo O(2^n)

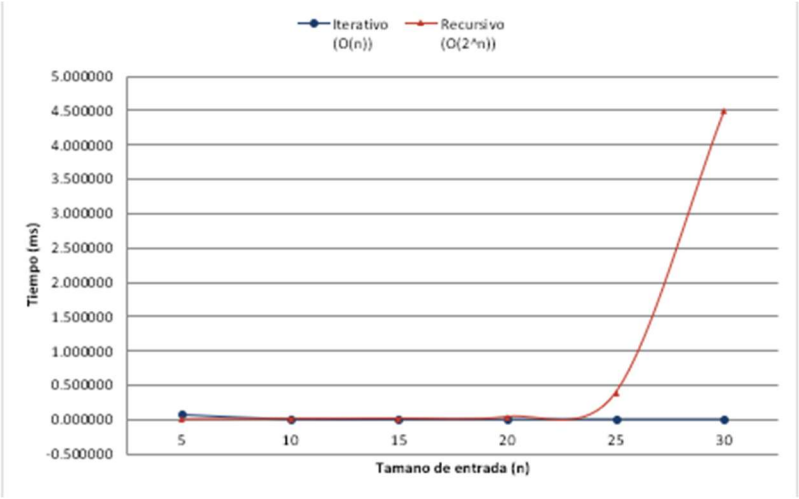
En el caso iterativo podemos ver reflejada la definición matemática mientras que el caso recursivo si bien es má eficiente ya que recalcula varias veces los mismos valores.

```
package algorithms;

public class Fibonacci {
   public Fibonacci() {
   }

   public static long iterativo(int var0) {
      if (var0 < 0) {
         throw new IllegalArgumentException("n no puede
ser negativo");
      } else if (var0 == 0) {
         return 0L;
```

```
      } else if (var0 == 1) {
         return 1L;
      } else {
         long var1 = 0L;
         long var3 = 1L;

         for(int var5 = 2; var5 <= var0; ++var5) {
            long var6 = var1 + var3;
            var1 = var3;
            var3 = var6;
         }

         return var3;
      }
   }

   public static long recursivo(int var0) {
      if (var0 < 0) {
         throw new IllegalArgumentException("n no puede
ser negativo");
      } else if (var0 > 30) {
         throw new IllegalArgumentException("Limita n ≤
30 en versión recursiva (O(2^n))");
      } else if (var0 == 0) {
         return 0L;
      } else {
         return var0 == 1 ? 1L : recursivo(var0 - 1) +
recursivo(var0 - 2);
      }
   }
}
```

n	Iterativo (O(n))	Recursivo (O(2^n))
5	0.077744	0.001298
10	0.000402	0.009784
15	0.000518	0.014230
20	0.000554	0.039068
25	0.000568	0.397872
30	0.000628	4.501338

Observacion: Recursivo crece exponencialmente: duplicar n casi cuadriplica el tiempo.



En esta ocasión podemos observar lo siguiente si bien el método recursivo inicio con una velocidad más baja que la iterativa, a medida que n va aumentando de tamaño, también lo hace el tiempo necesario para realizarlo, por lo que a diferencia del método iterativo que mantiene un tiempo relativamente similar podemos concluir que este ultimo es mucho mas eficiente en este caso.

Búsqueda lineal:

Busca un elemento recorriéndolo elemento por elemento, en el método iterativo utilizamos un ciclo for que compara cada posición.

En cuanto al método recursivo avanza llamada por llamada hasta llegar al valor o al final.

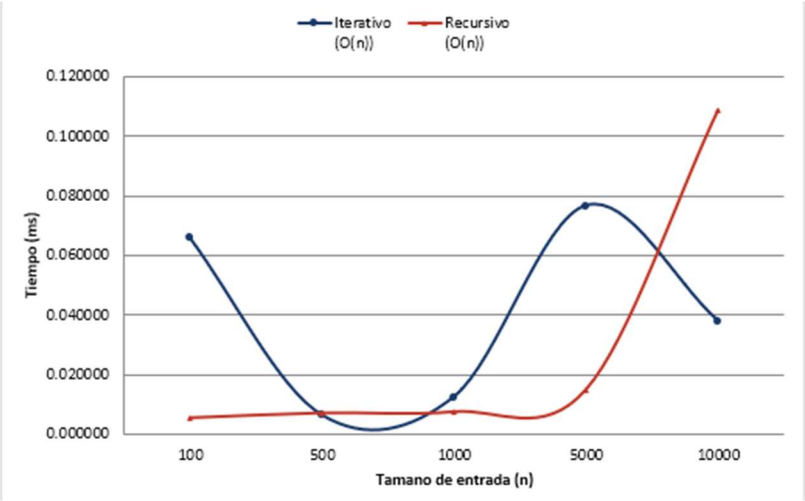
Este es un algoritmo simple y fácil de implementar, aunque es ineficiente en arreglos grandes.

```
package algorithms;

public class BusquedaLineal {

    // Versión iterativa O(n)
    public static int buscarIterativo(int[] arr, int
valor) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == valor)
                return i;
        }
        return -1;
```

```
    }

    // Versión recursiva O(n)
    public static int buscarRecursivo(int[] arr, int
valor, int indice) {
        if (indice >= arr.length)
            return -1;
        if (arr[indice] == valor)
            return indice;
        return buscarRecursivo(arr, valor, indice + 1);
    }
}
```

n	Iterativo (O(n))	Recursivo (O(n))
100	0.065730	0.005424
500	0.006290	0.007050
1000	0.012432	0.007458
5000	0.076648	0.014886
10000	0.037980	0.108650

Observacion: Ambas O(n): al multiplicar n x10, el tiempo tambien x10 aprox.



Analizando los resultados obtenidos por las distintas ejecuciones podemos observar que en promedio el método iterativo tiene mayor irregularidad en su velocidad oscilando sin importar si es valor n es mas grande o pequeño, a diferencia del recursivo que se mantiene en manera ascendente según crezca el valor de n, así crece también el tiempo necesario.

Ordenamiento burbuja:

En este algoritmo comparamos dos elementos uno al lado del otro para identificar si están en el orden correcto sino se intercambian de posición hasta que el arreglo este ordenado.

En el caso iterativo se utilizan dos ciclos anidados para recorrer y comparar los elementos.

En el caso recursivo reduce el tamaño del problema en cada llamada, ordenando paso a paso.

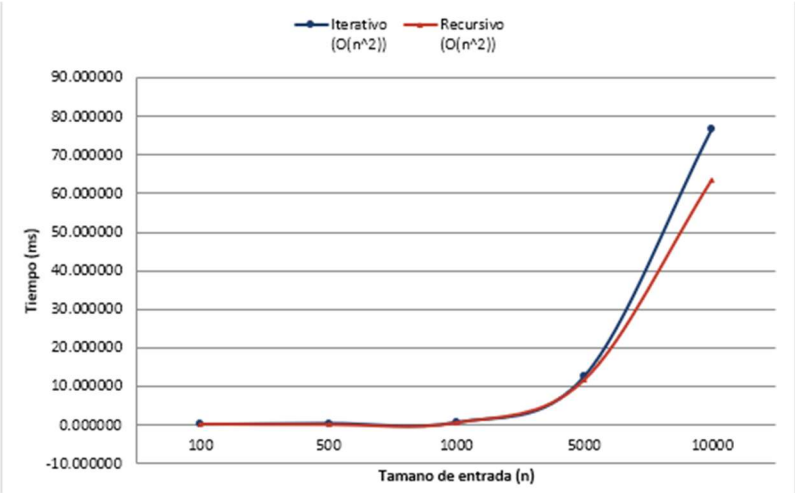
Su complejidad es O(n^2), es un algoritmo fácil de entender e implementar, pero es muy ineficiente con arreglos grandes.

```
package algorithms;

public class OrdenamientoBurbuja {

    // Versión iterativa O(n^2)
    public static void burbujaIterativo(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }

    // Versión recursiva O(n^2)
    public static void burbujaRecursivo(int[] arr, int
n) {
        if (n == 1)
            return;
        for (int i = 0; i < n - 1; i++) {
            if (arr[i] > arr[i + 1]) {
                int temp = arr[i];
                arr[i] = arr[i + 1];
                arr[i + 1] = temp;
            }
        }
```

```
        burbujaRecursivo(arr, n - 1);
    }
}
```

n	Iterativo (O(n^2))	Recursivo (O(n^2))
100	0.233676	0.258302
500	0.482068	0.148850
1000	0.706360	0.578058
5000	12.535072	11.658218
10000	76.787124	63.712130

Observacion: Ambas O(n^2): al x10 n, el tiempo aumenta aprox x100.



Ya que en ambos casos mantiene la misma complejidad su crecimiento es igualmente exponencial en ambos casos, aunque si debemos notar la diferencia clara que hay entre el tamaño el tiempo ya que en el recursivo se inicia con un poco más de tiempo, pero a medida que n crece, el aumento de tiempo es más elevado en la iterativa que en recursivo, y podemos intuir que a medida que el número siga creciendo la diferencia será aún mayor.

Conclusión:

Se implementaron 4 algoritmos en su versiones iterativas y recursivas, cada uno presenta características distintas en cuanto a eficiencia y claridad. La factorial y la búsqueda lineal tienen la misma complejidad O(n) lo que significa que su tiempo de ejecución crece proporcionalmente al tamaño de la entrada. Mientras que el ordenamiento burbuja tiene complejidad O(n^2), mostrando crecimiento cuadrático que lo hace poco eficiente. Estos ejemplos permiten comparar como la elección entre iteración y recursión afecta directamente el rendimiento de los algoritmos.